Learning Module: W6M1

March 15, 2026

Contents

1 1. Dataset Preparation & Practical Motivation **3**
1.1 Before We Begin . . . 3
1.2 Motivation . . . 3
1.3 Intuition . . . 3
1.4 Formal Definition . . . 3
1.5 Worked Example . . . 4
1.6 Key Takeaway . . . 5

2 2. Data Splitting: Train, Validation, and Test **12**
2.1 Before We Begin . . . 12
2.2 Motivation . . . 12
2.3 Intuition . . . 12
2.4 Formal Definition . . . 12
2.5 Worked Example . . . 13
2.6 Key Takeaway . . . 14
2.7 Questions . . . 14

3 End of Section **22**

4 3. Cross-Validation (CV) Techniques **23**
4.1 Before We Begin . . . 23
4.2 Motivation . . . 23
4.3 Intuition . . . 23
4.4 Formal Definition . . . 23
4.5 Worked Example . . . 24
4.6 Key Takeaway . . . 25

5 Limitations of Cross-Validation **31**

6 Handling Class Imbalance **40**
6.1 Theory Overview . . . 40
6.1.1 Balancing Techniques . . . 40
6.2 Worked Example Recap . . . 40
6.3 Key Takeaway . . . 41
6.4 Questions . . . 41
6.4.1 Recall . . . 41
6.4.2 Understand . . . 42
6.4.3 Apply . . . 43
6.4.4 Analyse . . . 44
6.4.5 Evaluate . . . 45

1 1. Dataset Preparation & Practical Motivation

1.1 Before We Begin

A dataset is a collection of data organized in a table. Each horizontal line of the table is called a row and represents one sample (for example, one patient). Each vertical line is a column and represents a feature (for example, age or blood pressure). One of the columns is often designated as the label or target that we want to predict (for example, whether a disease is present). Understanding these parts lets us see why we must look carefully at the data before building a model.

1.2 Motivation

example_first — Let's start with a concrete scenario. Preparing a dataset is like gathering and cleaning ingredients before cooking a meal: you first inspect each ingredient, discard any that are spoiled, and cut them to the right size before you can start cooking. Just as unclean ingredients can ruin a dish, raw data that contain missing entries, mislabeled rows, or hidden "fingerprints" can mislead a machine-learning model. By curating and cleaning the data we create a reliable foundation for training models that truly learn the patterns we care about.

1.3 Intuition

Think of dataset preparation as a careful inspection and organization step. First we glance through the whole table to get a feel for what values are present. Next we fix obvious problems such as empty cells (missing values) or obvious mistakes in the labels (label noise). We also make sure that the way we split the data for training and testing does not unintentionally share information between them, especially when the data come from a small pool of subjects. When these steps are done, the remaining data are ready to be handed to a learning algorithm with confidence that the algorithm is seeing clean, representative information.

1.4 Formal Definition

The preparation of a machine-learning dataset proceeds through four essential steps:

1. **Initial scan.** Examine every row and column to become familiar with the range of values, the presence of outliers, and the overall structure.

2. **Curation and handling of missing values.** Identify any cells that contain no value (a missing value). Decide on a strategy such as filling the cell with a plausible estimate (imputation) or removing the affected row, because algorithms generally cannot operate on empty entries.

3. **Denoising and label-noise correction.** Look for rows whose label (the target column) appears inconsistent with the feature values; these are instances of label noise. Correct the label when the true class can be inferred, because learning from incorrect targets corrupts the model.

4. **Ground-truth preparation.** Ensure that the final set of rows and columns truly represent the problem of interest, and verify that the splitting of data into training and testing subsets respects any special constraints (e.g., using subject-level splits for medical data) to avoid *data leakage*.

These steps address common misconceptions: missing values are not automatically handled by most algorithms, label noise is not harmless, and simply adding more data does not replace the need for careful curation.

1.5 Worked Example

Problem statement. We have a tiny CSV file with 10 rows. The columns are `Age`, `BloodPressure`, and the label `DiseasePresent`. We will walk through the four preparation steps and produce a clean version ready for training.

1. **Step 1 – Scan the file.** Reading the file shows the following rows (excerpt):

   ```
   Row 1: 45, 130, 1
   Row 2: 60, 140, 0
   Row 3: 52, ?, 1
   ...
   Row 7: 38, 120, 0
   Row 8: 47, 135, 1
   Row 9: 55, 128, 0
   Row 10: 49, 132, 1
   ```

   We notice a question mark "?" in Row3, Column `BloodPressure`, indicating a missing value. All other rows appear syntactically correct.

2. **Step 2 – Handle the missing value.** We decide to impute the missing blood pressure with the average of the observed `BloodPressure` values. Average = (130+140+120+135+128-
   = 131.17 (rounded to 131). We replace the "?" in Row3 with 131.

3. **Step 3 – Detect and fix label noise.** Row5 reads: `Age`=70, `BloodPressure`=145, `DiseasePresent`=0. Given the high age and blood pressure, the clinical rule of thumb suggests a high likelihood of disease, making the label suspect. We consult the original medical record and discover the label should be 1. We change the label in Row5 to 1.

4. **Step 4 – Prepare the final curated dataset.** The cleaned table now reads:

   ```
   45,130,1
   60,140,0
   52,131,1   % imputed value
   ...
   38,120,0
   47,135,1
   55,128,0
   49,132,1
   70,145,1   % corrected label
   ```

All missing entries have been filled, and the mislabeled row has been corrected. The dataset is now ready for splitting and model training.

What this shows us: Even a very small dataset can contain missing values and label mistakes; detecting and fixing them is straightforward but essential for trustworthy model results.

1.6 Key Takeaway

Preparing a dataset is a disciplined, step-by-step cleaning process that removes missing values, corrects label errors, and avoids data leakage—especially when the data are few or come from sensitive domains such as medicine.

Questions

1. (Recall) List and briefly describe the four essential steps involved in preparing a machine-learning dataset as presented in the section.

2. (Understand) In your own words, explain why splitting data at the subject level (rather than at the individual sample level) is particularly important when working with medical datasets.

3. (Understand) Describe a potential consequence for model evaluation if a medical dataset were split randomly at the sample level instead of the subject level.

4. (Apply – Numerical) Consider the following tiny dataset of five patients:

PatientID	Age	BloodPressure	DiseasePresent	
1	50	135	1	
2	62	?	0	
3	48	128	1	
4	55	?	0	
5	70	145	0	(suspected mislabeled)

   (a) Compute the imputed value for the missing BloodPressure entries using the average of the observed BloodPressure values.

   (b) After imputation, indicate how you would correct the mislabeled row.

   (c) Summarize the sequence of preparation steps you would perform on this dataset before feeding it to a learning algorithm.

5. (Apply) Given a small dataset that contains two missing feature values and one row with an incorrect label, outline the actions you would take (including decisions about imputation, removal, and label correction) to ready the data for modeling, referencing the four preparation steps from the theory.

6. (Analyze) Compare the likely impact on model performance of (a) ignoring the missing values entirely (e.g., dropping rows) versus (b) imputing them with the mean of the observed values. Discuss how each approach could affect bias and variance in the trained model.

7. (Evaluate) Critically assess the statement: “If a dataset is small, we can skip thorough curation because any model will overfit anyway.” Use concepts from the section to argue whether this reasoning is valid or flawed, and suggest what curation steps remain essential even for tiny datasets.

Notation Used in This Section

Symbol	Meaning
$\mathcal{D}_{\text{train}}$	Training dataset
$\mathcal{D}_{\text{val}}$	Validation dataset
$\mathcal{D}_{\text{test}}$	Test dataset
$\ell_{\text{train}}(t)$	Training loss at epoch t
$\ell_{\text{val}}(t)$	Validation loss at epoch t

2 2. Data Splitting: Train, Validation, and Test

2.1 Before We Begin

Overfitting happens when a model learns patterns that are specific to the training data but do not generalize to new data. An epoch is one complete pass through the entire set of training examples. Because a model has already seen the training examples, it can achieve a lower error on them than on data it has never encountered. Recognizing when performance on unseen data starts to degrade is essential for building reliable models. This understanding prepares us to discuss how we deliberately set aside portions of the data for different purposes.

2.2 Motivation

Splitting data is like a student preparing for a final exam by first studying (training set), then taking practice quizzes (validation set) to find weak spots, and finally sitting the real exam (test set) to see the true result. When all the data is used for learning, there is no independent check on whether the model will work on future cases. Creating separate subsets lets us monitor learning, choose the best model, and obtain an honest estimate of real-world performance.

2.3 Intuition

Think of the three subsets as three roles in a rehearsal. The **training set** is the rehearsal material that the model repeatedly practices on. The **validation set** is a friend who watches the rehearsal and points out mistakes, helping the director decide which scenes to keep or cut and which lighting (hyper-parameters) to adjust. Finally, the **test set** is the audience on opening night; their reaction shows how well the performance will be received in the real world. By keeping the audience separate from the rehearsal, we avoid the illusion that the play is perfect simply because the actors got used to the rehearsal stage.

2.4 Formal Definition

Let

$$\mathcal{D}_{\text{train}}$$

denote the training dataset, the portion of data used by the learning algorithm to adjust its internal parameters.

$$\mathcal{D}_{\text{val}}$$

denote the validation dataset, a held-out subset that is never used for parameter updates but is consulted during training to evaluate how the current model would behave on unseen data.

$$\mathcal{D}_{\text{test}}$$

denote the test dataset, which remains untouched until the final trained model is ready to be reported; its performance serves as a proxy for deployment.

During iterative training (e.g., stochastic gradient descent), the loss on the training set at epoch t is written as

$$\ell_{\text{train}}(t).$$

The loss measured on the validation set at the same epoch is

$$\ell_{\text{val}}(t).$$

Typically $\ell_{\text{train}}(t)$ declines monotonically because the model is explicitly optimized to reduce it. In contrast, $\ell_{\text{val}}(t)$ often follows a pattern: it decreases at first (the model is learning generalizable patterns), then reaches a minimum, and may start to increase when the model begins to capture noise specific to $\mathcal{D}_{\text{train}}$ – the hallmark of over-fitting. The point where $\ell_{\text{val}}(t)$ stops decreasing and begins to rise signals that further training is unlikely to improve, and may even hurt, performance on new data. Note that the validation set is not the same as the test set; only after the model has been selected based on $\mathcal{D}_{\text{val}}$ do we evaluate it once on $\mathcal{D}_{\text{test}}$. Adding more examples to $\mathcal{D}_{\text{train}}$ can help but does not guarantee the disappearance of overfitting, especially if the model capacity remains too high. Likewise, a continuously decreasing validation loss does not guarantee continued improvement; the loss curve must be inspected for a clear minimum.

2.5 Worked Example

Problem statement. Create a synthetic binary-classification dataset with 200 samples (features x_1, x_2 and label y). Split the data into 75

Step1 – Generate the data. For $i = 1, \ldots, 200$ draw $x_1^{(i)} \sim \mathcal{N}(0, 1)$ and $x_2^{(i)} \sim \mathcal{N}(0, 1)$. Assign a label by $y^{(i)} = \mathbf{1}\{\, x_1^{(i)} + x_2^{(i)} + \epsilon^{(i)} > 0 \,\}$ where $\epsilon^{(i)} \sim \mathcal{N}(0, 0.2)$. The resulting set of 200 triples $(x_1^{(i)}, x_2^{(i)}, y^{(i)})$ forms the full dataset $\mathcal{D}$.

Step2 – Split the data. 751510The split is performed randomly but kept fixed for the whole experiment.

Step3 – Train and record losses. Using stochastic gradient descent with learning rate 0.1, iterate for epochs $t = 1, \ldots, 20$. At each epoch compute the average cross-entropy loss on $\mathcal{D}_{\text{train}}$ (giving $\ell_{\text{train}}(t)$) and on $\mathcal{D}_{\text{val}}$ (giving $\ell_{\text{val}}(t)$). The computed values are:

t	$\ell_{\text{train}}(t)$	$\ell_{\text{val}}(t)$
1	0.693	0.692
2	0.658	0.652
3	0.624	0.618
4	0.592	0.589
5	0.562	0.562
6	0.534	0.540
7	0.508	0.522
8	0.484	0.509
9	0.462	0.500
10	0.441	0.496
11	0.422	0.498
12	0.404	0.503
13	0.387	0.511
14	0.371	0.522
15	0.356	0.535
16	0.342	0.550
17	0.329	0.567
18	0.317	0.585
19	0.306	0.604
20	0.295	0.624

Step4 – Plot the curves. A line plot with epoch t on the horizontal axis and loss on the vertical axis shows two lines: the blue line (training loss) falling steadily, the orange line (validation loss) decreasing up to epoch10 and then rising afterwards.

Step5 – Identify the over-fitting point. The validation loss reaches its minimum at epoch10 ($\ell_{\text{val}}(10) = 0.496$). From epoch11 onward the validation loss increases, indicating that the model begins to overfit the training data after epoch10.

Final answer. The model starts to overfit after epoch10; stopping training at epoch10 would give the best generalization according to the validation set.

What this shows us: A validation set provides a concrete signal (the rise of $\ell_{\text{val}}(t)$) that tells us when further training harms the ability to generalize, even though the training loss $\ell_{\text{train}}(t)$ continues to improve.

2.6 Key Takeaway

Use three distinct data splits—training, validation, and test—to train, tune, and finally assess a model; the validation loss curve is the practical tool that reveals when overfitting begins.

2.7 Questions

Recall 1. According to the material, what is the commonly used percentage split of a dataset into training, validation, and test sets?

Understand 1. Explain why a separate validation set is kept apart from the training data during the model-development process. Include in your answer the role it plays with respect to overfitting and model selection.

Understand 2. Describe how the validation loss curve can be used to decide when to stop training a model. What pattern in the curve signals that further training may be harmful?

Apply 1. The following validation losses were recorded for epochs1 through10:

Epoch t	$\ell_{\text{val}}(t)$
1	0.692
2	0.652
3	0.618
4	0.589
5	0.562
6	0.540
7	0.522
8	0.509
9	0.500
10	0.496

Using these values, after which epoch does the model begin to overfit the training data? Justify your answer by referring to the behaviour of the validation loss.

Apply 2. Based on the same loss table, at which epoch does the validation loss attain its minimum value? If you were to employ early-stopping using the validation set, on which epoch would you stop training?

Apply 3. Compute the difference between the training loss $\ell_{\text{train}}(t)$ and the validation loss $\ell_{\text{val}}(t)$ at epoch5, using the training loss $\ell_{\text{train}}(5) = 0.562$ from the worked example. What does the magnitude of this difference suggest about the model's generalisation at that point?

Analyse 1. Discuss how increasing or decreasing the proportion of data allocated to the validation set (while keeping the total dataset size fixed) might influence the reliability of hyper-parameter tuning. Consider the trade-offs between having enough data to obtain a stable estimate of validation loss and retaining sufficient training data for learning.

Evaluate 1. Critically assess the following claim: "If the validation loss never rises, the model will perform perfectly on unseen data." In your answer, refer to the concepts of overfitting, the distinction between validation and test sets, and any limitations of relying solely on the validation loss trajectory.

3 End of Section

Notation Used in This Section

Symbol	Meaning
K	Number of folds in K-fold CV
$\mathcal{D}_{\text{train}}^{(k)}$	Training data for fold k
$\mathcal{D}_{\text{val}}^{(k)}$	Validation data for fold k
$\hat{y}^{(k)}$	Prediction made by the model trained on fold k

4 3. Cross-Validation (CV) Techniques

4.1 Before We Begin

A "fold" is simply a subset of the whole data set that we temporarily set aside for testing while the remaining parts are used for training. When we repeat a train-test experiment with different folds, each run gives its own performance number. Averaging those numbers reduces the chance that an unlucky random split will mislead us. Randomness in a single split can make a model look better or worse than it truly is. Understanding this repetition prepares us to see why cross-validation is useful for small data sets.

4.2 Motivation

Cross-validation is like a rotating tasting panel at a restaurant: each night a different group of diners samples the menu, and over the week every dish gets evaluated by every panel, giving a balanced view of quality. When a data set is tiny, a single train-test split may either exaggerate performance or waste valuable training material. By systematically rotating which samples are held out, we obtain a more reliable estimate of how the model will behave on unseen cases.

4.3 Intuition

Imagine you have a handful of cards and you want to know how well a magic trick works. Instead of showing the trick with just one random half of the cards, you repeatedly pick a different half, perform the trick, and record the success each time. Because every card gets a chance to be in the "test" pile, the overall success rate reflects the true power of the trick rather than the luck of one particular split. Cross-validation applies this idea to machine-learning models, giving each observation a turn to be evaluated while still using most data for learning.

4.4 Formal Definition

Let K denote the number of folds we decide to create. For each fold indexed by $k \in \{1, \ldots, K\}$ we separate the data into a training portion $\mathcal{D}_{\text{train}}^{(k)}$ and a validation portion $\mathcal{D}_{\text{val}}^{(k)}$. A model trained on $\mathcal{D}_{\text{train}}^{(k)}$ produces predictions $\hat{y}^{(k)}$ for the samples in $\mathcal{D}_{\text{val}}^{(k)}$. The performance (e.g., accuracy) measured on $\mathcal{D}_{\text{val}}^{(k)}$ is recorded for that fold, and the overall estimate is the average of the K recorded performances, i.e. $\frac{1}{K}\sum_{k=1}^{K} \text{performance}^{(k)}$. In plain K-fold cross-validation the folds are formed arbitrarily; in stratified K-fold each $\mathcal{D}_{\text{val}}^{(k)}$

preserves the same class proportion as the whole data set, which is especially important when some classes are rare. Leave-One-Out (LOO) CV is the extreme case $K = N$ where N is the total number of samples, so each fold holds out a single observation; this gives the most exhaustive use of data but requires training the model N times, which can be computationally heavy. Leave-P-Out (LPO) CV holds out every possible combination of p samples, leading to $\binom{N}{p}$ trainings and quickly becoming infeasible for any modest N.

4.5 Worked Example

Problem statement: A synthetic binary data set contains 10 samples, 5 belonging to classA and 5 to classB. We will perform a 5-fold stratified cross-validation, compute the accuracy for each fold, obtain the average accuracy, and finally illustrate how the five trained models could vote on a new sample.

Step1 – Create the folds (stratified). Because the data are balanced, each fold must contain exactly oneA and oneB sample. The folds are:

Fold k	$\mathcal{D}_{\text{val}}^{(k)}$	$\mathcal{D}_{\text{train}}^{(k)}$
1	$\{A_1,\ B_1\}$	$\{A_2, A_3, A_4, A_5,\ B_2, B_3, B_4, B_5\}$
2	$\{A_2,\ B_2\}$	$\{A_1, A_3, A_4, A_5,\ B_1, B_3, B_4, B_5\}$
3	$\{A_3,\ B_3\}$	$\{A_1, A_2, A_4, A_5,\ B_1, B_2, B_4, B_5\}$
4	$\{A_4,\ B_4\}$	$\{A_1, A_2, A_3, A_5,\ B_1, B_2, B_3, B_5\}$
5	$\{A_5,\ B_5\}$	$\{A_1, A_2, A_3, A_4,\ B_1, B_2, B_3, B_4\}$

Step2 – Train and evaluate each fold. Assume a simple classifier that perfectly separates the two classes when both are present in the training set. Because each training set contains all remaining A and B samples, the model predicts the two validation samples correctly.

Thus, for every fold k the number of correct predictions is 2 out of 2, giving an accuracy of

$$\text{accuracy}^{(k)} = \frac{2}{2} = 1.0 \quad \text{for } k = 1, \ldots, 5.$$

Step3 – Compute the average accuracy.

$$\text{average accuracy} = \frac{1}{K}\sum_{k=1}^{K} \text{accuracy}^{(k)} = \frac{1}{5}\left(1.0 + 1.0 + 1.0 + 1.0 + 1.0\right) = 1.0.$$

Step4 – Ensemble voting for a new sample. Each of the five models (trained on the five different training folds) predicts a label for a new, unseen sample. Suppose their predictions are:

$$\hat{y}^{(1)} = \text{A},\ \hat{y}^{(2)} = \text{A},\ \hat{y}^{(3)} = \text{B},\ \hat{y}^{(4)} = \text{A},\ \hat{y}^{(5)} = \text{B}.$$

A majority-vote ensemble selects the label occurring most often, which is classA (three votes).

What this shows us: Stratified 5-fold CV uses all data efficiently, yields an unbiased estimate (here perfect accuracy because the problem is easy), and the five models can be combined by majority voting to make a robust prediction on new data.

4.6 Key Takeaway

Cross-validation repeatedly partitions the data so every observation is tested once, providing a more trustworthy performance estimate—especially when data are scarce or class distributions are uneven.

Questions

1. **Recall:** Define what a "fold" is in the context of K-fold cross-validation. (Answer in one sentence, using the terminology from the theory section).

2. **Understand:** Explain in your own words why stratified K-fold cross-validation is preferable when the data set is imbalanced. Illustrate your explanation with reference to class proportions.

3. **Understand:** Suppose you accidentally create ordinary (non-stratified) 4-fold splits on a data set where class0 represents 10

4. **Apply:** A data set contains 12 samples with a class ratio of 3:1 (i.e., 9 samples of class0 and 3 samples of class1). Design a 3-fold stratified split. For each fold list how many class-0 and class-1 samples appear in the validation (test) part of the fold.

5. **Apply (Numerical):** Using the 5-fold stratified cross-validation described in the worked example (10 samples, 5 of each class) assume a classifier obtains the following accuracies on the five folds: 1.0, 0.9, 1.0, 0.8, 1.0. Compute the overall average accuracy that would be reported for the whole cross-validation procedure.

6. **Analyse:** Compare the computational cost of performing a 5-fold cross-validation versus a Leave-One-Out cross-validation on a data set containing 100 samples. Discuss how the number of model trainings differs between the two approaches and what this implies for runtime.

7. **Evaluate:** Critique the following statement: "Using more folds always yields a more reliable estimate of model performance." In your answer, consider both the potential benefits and the possible drawbacks, referring to concepts such as bias, variance, and computational expense.

5 Limitations of Cross-Validation

Training a model many times means repeating the whole learning process, which multiplies the amount of computation needed. Each additional training run consumes more processor cycles and memory, lengthening the overall runtime. When the data used for evaluation contain systematic errors or a skewed representation of the real-world population, the performance numbers we obtain can be misleading. These two ideas — repeated training increasing cost, and biased data distorting evaluation — set the stage for understanding the limits of cross-validation.

Testing a car's performance by repeatedly driving it on a narrow, winding track is useful, but if the track is too short or the same obstacle appears every lap, the test will never reveal how the car behaves on a highway. Cross-validation was created to obtain a more reliable estimate of how a model will perform on unseen data, but the procedure itself can introduce new problems. Recognizing those problems helps us decide when cross-validation is appropriate and when alternative strategies are needed.

Think of cross-validation as repeatedly sampling different portions of a dataset to train and test a model. If the sampling process is costly, if the data contain a lot of random noise, or if the sampled portions do not represent the true population, the repeated tests will give a picture that looks better than reality. In other words, the method can be expensive, can still over-fit to quirks in the data, and can produce overly optimistic scores when the underlying data are biased or when the folds are poorly constructed.

Cross-validation suffers from four major limitations:

1. **Computationally Expensive.** The procedure requires training the model K times for K-fold cross-validation (or N times for leave-one-out cross-validation, where N is the number of samples). Each training run repeats the full learning algorithm, so the total runtime grows roughly in proportion to the number of folds.

2. **Overfitting to Noise.** When the dataset contains a high level of random variation or outliers, the repeated fitting can capture those idiosyncrasies rather than the underlying pattern, leading to a model that appears accurate on the cross-validation splits but performs poorly on truly new data.

3. **Optimistic Bias.** If the data used for cross-validation do not faithfully represent the real-world population—because of sampling bias, measurement bias, or other systematic distortions—the performance estimates will be systematically higher than what would be observed in practice.

4. **Fold Selection Bias.** When the data are divided into folds without respecting important structure (for example, not using stratified sampling on an imbalanced binary outcome, or not grouping by patient in a medical study), some folds may contain unrepresentative class proportions or leakage of subject-level information, causing the reported metrics to be misleading.

These points address common misconceptions: cross-validation does not magically remove all sources of bias, increasing the number of folds does not reduce computational effort, and random assignment of samples to folds does not guarantee the absence of leakage or bias.

Cross-validation provides useful performance estimates, but it can be costly, still over-fit to noisy or biased data, and produce overly optimistic results when folds are not constructed carefully.

Questions

Recall: List and briefly describe the four limitations of cross-validation that are discussed in this section.

Understand: In your own words, explain why using a dataset that does not represent the real-world population can cause cross-validation to produce overly optimistic performance estimates.

Understand: Describe how repeated training of a model during cross-validation contributes to computational expense, and why this expense does not diminish when the number of folds is increased.

Apply: A 5-fold cross-validation on a dataset containing 1,000 samples takes 30 minutes to complete. Assuming that the runtime scales linearly with the number of folds, estimate how long a 10-fold cross-validation on the same dataset would take.

Apply: For the same 1,000-sample dataset, leave-one-out cross-validation (LOO) requires training the model once for each sample. If a single training run takes 0.5 minutes, calculate the total runtime required for LOO CV and compare it with the 30-minute runtime of the 5-fold CV.

Analyse: Consider an imbalanced binary classification problem where the minority class makes up only 10

Evaluate: Judge the following statement:
"If I have enough computing power, I can ignore all limitations of cross-validation."
Provide a reasoned argument supporting or refuting the statement, referring to the limitations covered in this section.

Analyse: Provide an example scenario where a highly noisy training set leads cross-validation to over-fit to random noise, thereby suggesting a deceptively high accuracy. Explain how techniques such as regularization, robust loss functions, or noise-aware cross-validation strategies can mitigate this issue.

Notation Used in This Section

Symbol	Meaning
N_{maj}	Number of majority-class samples
N_{min}	Number of minority-class samples
w_{min}	Weight or cost assigned to minority-class errors in cost-sensitive learning

6 Handling Class Imbalance

Class imbalance arises when the classes in a classification problem occur with drastically different frequencies. Most learning algorithms are designed for roughly balanced data; when one class overwhelms the other, the model may become biased towards the majority, leading to poor detection of rare but important events such as a disease or fraud. To mitigate this, several rebalancing strategies can be applied either to the data itself or to the learning objective.

6.1 Theory Overview

6.1.1 Balancing Techniques

- **Oversampling**. Duplicate minority samples until their count approaches that of the majority.
- **Undersampling**. Randomly discard majority samples until the two classes are roughly equal.
- **Data Augmentation**. Apply small label-preserving transformations (e.g. rotations of images) to minority samples to create new, slightly different examples.
- **SMOTE** (Synthetic Minority Over–sampling Technique). For each minority instance, interpolate with its nearest minority neighbors to generate synthetic points that are plausible but not exact copies.
- **Cost-Sensitive Learning**. Modify the loss function by assigning a higher penalty to misclassifying minority instances. Denote this penalty by w_{min}.

When using cross-validation, rebalancing must be done only on the training folds; validation and test folds should stay in their natural, imbalanced state to provide an honest performance estimate.

6.2 Worked Example Recap

A binary dataset contains 200 samples: 180 majority class (0) and 20 minority class (1). After holding out 40 samples (30 majority, 10 minority) for testing, the remaining 160 samples form the training pool. Three pipelines are compared:

1. Raw data (144 majority, 16 minority).
2. Oversample to 144 minority copies (288 total).

3. SMOTE to 160 synthetic minorities (304 total).

All pipelines achieve 80% accuracy, but the balanced approaches substantially improve recall and F1 for the minority class, illustrating the benefit of rebalancing.

6.3 Key Takeaway

Balancing is essential for training but should never be applied to evaluation data. The goal is to build a classifier that respects the real, often imbalanced distribution of the target variable.

6.4 Questions

6.4.1 Recall

Recall 1. What are the two basic strategies for rebalancing an imbalanced dataset?

Recall 2. Name the technique that creates new, plausible minority–class examples by interpolating between existing minority samples, instead of simply duplicating them.

6.4.2 Understand

Understand 1. Explain in your own words why balancing techniques (e.g., oversampling, undersampling, SMOTE) must be applied only to the training folds when performing K-Fold cross-validation.

Understand 2. Suppose you also applied an oversampling step to the validation folds. What potential problem could this introduce when you later evaluate model performance on a truly imbalanced test set?

6.4.3 Apply

Apply 1. You have a training set containing 1,000 majority-class samples and 200 minority-class samples. Using simple oversampling (i.e., duplicating minority instances), how many additional minority samples must you generate so that the training set becomes perfectly balanced (1:1 ratio)?

Apply 2. In the worked example, the training pool after the hold-out step contains 16 minority samples. If you apply SMOTE and create 5 synthetic samples for each original minority instance, how many minority samples will the training set contain after this step?

Apply 3. For a cost-sensitive classifier you decide to set the minority-class penalty $w_{\min} = 3$ while keeping the majority-class penalty equal to 1. Briefly describe how this weighting will affect the loss function during training.

6.4.4 Analyse

Analyse 1. Compare the potential impact on model bias of (a) oversampling the minority class and (b) using a cost-sensitive learning approach. Discuss how each method influences the classifier's tendency to favour the majority class.

Analyse 2. Considering the pipelines (a) raw data, (b) oversampling, and (c) SMOTE from the worked example, explain why the overall accuracy remained the same across all three pipelines while recall and F1-score for the minority class improved.

6.4.5 Evaluate

Evaluate 1. Critically assess the claim:

"Undersampling is always preferable because it reduces training time without affecting model performance."

In your answer, consider (i) the effect of discarding majority-class information, (ii) potential changes in model bias and variance, and (iii) situations where undersampling might actually hurt performance.